IPNS 이름 시스템

개요

IPNS (InterPlanetary Name System)는 IPFS의 가변 이름 시스템으로, 불변의 IPFS 해시를 사람이 읽기 쉬운 이름이나 업데이트 가능한 포인터로 매핑할 수 있게 해줍니다. IPNS를 통해 콘텐츠가 변경되어도 동일한 이름으로 접근할 수 있습니다.

IPNS는 공개키 암호화를 사용하여 레코드의 진위성을 보장하며, 각 IPNS 이름은 공개키의 해시로 생성됩니다. 이를 통해 해당 개인키를 가진 사람만이 레코드를 업데이트할 수 있습니다.

IPNS 개념과 레코드 구조

IPNS 이름 (Name)

IPNS 이름은 공개키의 멀티해시로 구성되며, 다음과 같은 형태를 가집니다:

```
/ipns/k51qzi5uqu5dgutdk6zqfxs1pnjz04yza9xp7rt5tbp9nqg3r9flvv5gqd5tnc
```

IPNS 레코드 구조

IPNS 레코드는 다음과 같은 필드들을 포함합니다:

- **Value**: 가리키는 IPFS 경로 (예: `/ipfs/QmHash...`)
- **Sequence**: 레코드의 버전 번호 (단조 증가)
- **Validity**: 레코드의 유효 기간 (EOL - End of Life)
- **TTL**: 캐시 유지 시간
- **Signature**: 레코드의 디지털 서명

기본 사용법

IPNS 이름 생성

```
package main

import (
"crypto/rand"
"fmt"
"log"
"github.com/ipfs/boxo/ipns"
"github.com/libp2p/go-libp2p/core/crypto"
"github.com/libp2p/go-libp2p/core/peer"
)

func  createIPNSName() {
// Ed25519 개인키 생성
sk, pk, err := crypto.GenerateEd25519Key(rand.Reader)
if  err != nil {
log.Fatal("키 생성 실패:", err)
}
// 피어 ID 생성
pid, err := peer.IDFromPublicKey(pk)
```

```
if err != nil {
log.Fatal("피어 ID 생성 실패:", err)
}
// IPNS 이름 생성
name := ipns.NameFromPeer(pid)
fmt.Printf("개인키: %s\n", sk)
fmt.Printf("공개키: %s\n", pk)
fmt.Printf("피어 ID: %s\n", pid)
fmt.Printf("IPNS 이름: %s\n", name.String())
fmt.Printf("IPNS 경로: %s\n", name.AsPath().String())
}
```

IPNS 레코드 생성

```
import (
"time"
"github.com/ipfs/boxo/ipns"
"github.com/ipfs/boxo/path"
)

func createIPNSRecord() {
// 개인키 생성
sk, _, err := crypto.GenerateEd25519Key(rand.Reader)
```

```
if err != nil {
log.Fatal("키 생성 실패:", err)
}
// 가리킬 IPFS 경로 설정
targetPath, err := path.NewPath("/ipfs/bafkqac3jobxhgidsn5rww4yk")
if err != nil {
log.Fatal("경로 생성 실패:", err)
}
// 레코드 유효 기간 설정 (1시간 후 만료)
eol := time.Now().Add(time.Hour)
// TTL 설정 (20초)
ttl := 20 * time.Second
// 시퀀스 번호 (첫 번째 레코드는 1)
sequence := uint64(1)
// IPNS 레코드 생성
record, err := ipns.NewRecord(sk, targetPath, sequence, eol, ttl)
if err != nil {
log.Fatal("레코드 생성 실패:", err)
}
// 레코드 직렬화
recordData, err := ipns.MarshalRecord(record)
if err != nil {
log.Fatal("레코드 직렬화 실패:", err)
```

```
}

fmt.Printf("IPNS 레코드 생성 완료 (크기: %d bytes)\n", len(recordData))

// 레코드 정보 출력

value, _ := record.Value()

validity, _ := record.Validity()

seq, _ := record.Sequence()

recordTTL, _ := record.TTL()

fmt.Printf("값: %s\n", value.String())

fmt.Printf("유효 기간: %s\n", validity.Format(time.RFC3339))

fmt.Printf("시퀀스: %d\n", seq)

fmt.Printf("TTL: %s\n", recordTTL)

}
```

IPNS 레코드 검증

```
func validateIPNSRecord(recordData []byte, name ipns.Name) error {

// 레코드 역직렬화

record, err := ipns.UnmarshalRecord(recordData)

if err != nil {

return fmt.Errorf("레코드 역직렬화 실패: %w", err)

}

// 이름과 함께 검증

err = ipns.ValidateWithName(record, name)
```

```
if err != nil {
return fmt.Errorf("레코드 검증 실패: %w", err)
}
fmt.Println("IPNS 레코드 검증 성공!")
return nil
}
```

이름 생성 및 검증 방법

다양한 키 타입으로 IPNS 이름 생성

```
func createIPNSNamesWithDifferentKeys() {
// Ed25519 키 (권장)
ed25519SK, ed25519PK, _ := crypto.GenerateEd25519Key(rand.Reader)
ed25519Name := ipns.NameFromPeer(mustPeerID(ed25519PK))
// RSA 키
rsaSK, rsaPK, _ := crypto.GenerateRSAKeyPair(2048, rand.Reader)
rsaName := ipns.NameFromPeer(mustPeerID(rsaPK))
// ECDSA 키
ecdsaSK, ecdsaPK, _ := crypto.GenerateECDSAKeyPair(rand.Reader)
ecdsaName := ipns.NameFromPeer(mustPeerID(ecdsaPK))
fmt.Printf("Ed25519 IPNS 이름: %s\n", ed25519Name.String())
fmt.Printf("RSA IPNS 이름: %s\n", rsaName.String())
```

```
fmt.Printf("ECDSA IPNS 이름: %s\n", ecdsaName.String())

// 키 타입별 레코드 생성 (공개키 임베딩 여부 확인)

testPath, _ := path.NewPath("/ipfs/bafkqac3jobxhgidsn5rww4yk")

eol := time.Now().Add(time.Hour)

ttl := 20 * time.Second

// Ed25519는 공개키를 임베드하지 않음 (피어 ID에서 추출 가능)

ed25519Record, _ := ipns.NewRecord(ed25519SK, testPath, 1, eol, ttl)

// RSA는 공개키를 임베드함 (피어 ID에서 추출 불가능)

rsaRecord, _ := ipns.NewRecord(rsaSK, testPath, 1, eol, ttl)

// 공개키 임베딩 여부 확인

_, ed25519PkErr := ed25519Record.PubKey()

_, rsaPkErr := rsaRecord.PubKey()

fmt.Printf("Ed25519 레코드 공개키 임베딩: %v\n", ed25519PkErr == nil)

fmt.Printf("RSA 레코드 공개키 임베딩: %v\n", rsaPkErr == nil)

}



func  mustPeerID(pk crypto.PubKey) peer.ID {

pid, err := peer.IDFromPublicKey(pk)

if  err != nil {

panic(err)

}

return  pid
```

```
}
```

문자열에서 IPNS 이름 파싱

```
func parseIPNSNames() {
// 다양한 형태의 IPNS 이름 파싱
examples := []string{
"k51qzi5uqu5dgutdk6zqfxs1pnjz04yza9xp7rt5tbp9nqg3r9flvv5gqd5tnc",
"/ipns/k51qzi5uqu5dgutdk6zqfxs1pnjz04yza9xp7rt5tbp9nqg3r9flvv5gqd5tnc",
"12D3KooWGC6TvWhfapngX6wvJHMYVKpDMXGb9ZJ2E5EHcUGBfMu8", // 피어 ID 형태
}
for _, example := range examples {
name, err := ipns.NameFromString(example)
if err != nil {
fmt.Printf("파싱 실패 '%s': %v\n", example, err)
continue
}
fmt.Printf("원본: %s\n", example)
fmt.Printf("IPNS 이름: %s\n", name.String())
fmt.Printf("피어 ID: %s\n", name.Peer())
fmt.Printf("CID: %s\n", name.Cid())
fmt.Printf("경로: %s\n", name.AsPath().String())
fmt.Println("---")
```

```
}

}
```

CID에서 IPNS 이름 생성

```
import (
"github.com/ipfs/go-cid"
"github.com/multiformats/go-multicodec"
)

func  createIPNSNameFromCID() {
// libp2p-key 코덱을 사용하는 CID 생성 (예시)
// 실제로는 공개키의 멀티해시를 사용해야 함
sk, pk, _ := crypto.GenerateEd25519Key(rand.Reader)
pid, _ := peer.IDFromPublicKey(pk)
// 피어 ID를 CID로 변환
c := cid.NewCidV1(cid.Libp2pKey, pid)
// CID에서 IPNS 이름 생성
name, err := ipns.NameFromCid(c)
if  err != nil {
log.Fatal("CID에서 IPNS 이름 생성 실패:", err)
}
fmt.Printf("CID: %s\n", c)
```

```
fmt.Printf("IPNS 이름: %s\n", name.String())

// 검증: 원래 키로 레코드 생성 및 검증

testPath, _ := path.NewPath("/ipfs/bafkqac3jobxhgidsn5rww4yk")

eol := time.Now().Add(time.Hour)

ttl := 20 * time.Second

record, _ := ipns.NewRecord(sk, testPath, 1, eol, ttl)

err = ipns.ValidateWithName(record, name)

if  err != nil {

log.Fatal("검증 실패:", err)

}

fmt.Println("CID에서 생성한 IPNS 이름 검증 성공!")

}
```

실제 사용 예제와 모범 사례

IPNS 레코드 관리자

```
package main


import (

"context"

"fmt"

"sync"
```

```
"time"

"github.com/ipfs/boxo/ipns"

"github.com/ipfs/boxo/path"

"github.com/libp2p/go-libp2p/core/crypto"

)


// IPNSManager는 IPNS 레코드의 생성, 업데이트, 검증을 관리합니다

type IPNSManager struct {

privateKey crypto.PrivKey

name ipns.Name

sequence  uint64

mu sync.RWMutex

}


func  NewIPNSManager(sk crypto.PrivKey) (*IPNSManager, error) {

pk := sk.GetPublic()

pid, err := peer.IDFromPublicKey(pk)

if  err != nil {

return  nil, err

}

return &IPNSManager{

privateKey: sk,

name: ipns.NameFromPeer(pid),
```

```
sequence: 0,

}, nil

}


func (m *IPNSManager) GetName() ipns.Name {

return  m.name

}


func (m *IPNSManager) CreateRecord(targetPath path.Path, ttl time.Duration) (*ip

m.mu.Lock()

defer  m.mu.Unlock()

// 시퀀스 번호 증가

m.sequence++

// 유효 기간 설정 (TTL의 10배 또는 최소 1시간)

validityDuration := ttl * 10

if  validityDuration < time.Hour {

validityDuration = time.Hour

}

eol := time.Now().Add(validityDuration)

// 레코드 생성

record, err := ipns.NewRecord(m.privateKey, targetPath, m.sequence, eol, ttl)

if  err != nil {

m.sequence-- // 실패 시 시퀀스 롤백
```

```
return nil, err
}
return record, nil
}


func (m *IPNSManager) UpdateRecord(newPath path.Path, ttl time.Duration) (*ipns.
record, err := m.CreateRecord(newPath, ttl)
if err != nil {
return nil, nil, err
}
// 레코드 직렬화
recordData, err := ipns.MarshalRecord(record)
if err != nil {
return nil, nil, err
}
// 자체 검증
err = ipns.ValidateWithName(record, m.name)
if err != nil {
return nil, nil, fmt.Errorf("생성된 레코드 검증 실패: %w", err)
}
return record, recordData, nil
}
```

```go
func (m *IPNSManager) GetCurrentSequence() uint64 {
m.mu.RLock()
defer m.mu.RUnlock()
return m.sequence
}

func (m *IPNSManager) ValidateRecord(recordData []byte) error {
record, err := ipns.UnmarshalRecord(recordData)
if err != nil {
return err
}
return ipns.ValidateWithName(record, m.name)
}

// 사용 예제
func ipnsManagerExample() {
// 개인키 생성
sk, _, err := crypto.GenerateEd25519Key(rand.Reader)
if err != nil {
log.Fatal(err)
}
// IPNS 매니저 생성
manager, err := NewIPNSManager(sk)
```

```
if err != nil {
log.Fatal(err)
}
fmt.Printf("IPNS 이름: %s\n", manager.GetName().String())
// 첫 번째 레코드 생성
path1, _ := path.NewPath("/ipfs/bafkqac3jobxhgidsn5rww4yk")
record1, data1, err := manager.UpdateRecord(path1, 30*time.Second)
if err != nil {
log.Fatal(err)
}
value1, _ := record1.Value()
seq1, _ := record1.Sequence()
fmt.Printf("레코드 1 - 값: %s, 시퀀스: %d\n", value1.String(), seq1)
// 두 번째 레코드 생성 (업데이트)
time.Sleep(1 * time.Second) // 시간 차이를 위해
path2, _ := path.NewPath("/ipfs/bafkreifjjcie6lypi6ny7amxnfftagclbuxndqonfipmb64
record2, data2, err := manager.UpdateRecord(path2, 30*time.Second)
if err != nil {
log.Fatal(err)
}
value2, _ := record2.Value()
seq2, _ := record2.Sequence()
fmt.Printf("레코드 2 - 값: %s, 시퀀스: %d\n", value2.String(), seq2)
```

```
// 레코드 검증

fmt.Println("레코드 1 검증:", manager.ValidateRecord(data1))

fmt.Println("레코드 2 검증:", manager.ValidateRecord(data2))

}
```

IPNS 레코드 비교 및 선택

```
func  recordComparisonExample() {

// 두 개의 다른 키로 레코드 생성

sk1, _, _ := crypto.GenerateEd25519Key(rand.Reader)

sk2, _, _ := crypto.GenerateEd25519Key(rand.Reader)

manager1, _ := NewIPNSManager(sk1)

manager2, _ := NewIPNSManager(sk2)

testPath, _ := path.NewPath("/ipfs/bafkqac3jobxhgidsn5rww4yk")

// 같은 이름에 대한 여러 레코드 생성 (시뮬레이션)

record1, data1, _ := manager1.UpdateRecord(testPath, 30*time.Second)

// 1초 후 더 높은 시퀀스로 업데이트

time.Sleep(1 * time.Second)

record2, data2, _ := manager1.UpdateRecord(testPath, 30*time.Second)

// 더 긴 유효 기간을 가진 레코드

record3, data3, _ := manager1.CreateRecord(testPath, 60*time.Second)

// Validator를 사용한 레코드 선택

validator := ipns.Validator{}
```

```
// 여러 레코드 중 최적 선택
records := [][]byte{data1, data2, data3}
selectedIndex, err := validator.Select(manager1.GetName().RoutingKey(), records)
if  err != nil {
log.Fatal("레코드 선택 실패:", err)
}
selectedRecord, _ := ipns.UnmarshalRecord(records[selectedIndex])
selectedValue, _ := selectedRecord.Value()
selectedSeq, _ := selectedRecord.Sequence()
selectedValidity, _ := selectedRecord.Validity()
fmt.Printf("선택된 레코드 (인덱스 %d):\n", selectedIndex)
fmt.Printf(" 값: %s\n", selectedValue.String())
fmt.Printf(" 시퀀스: %d\n", selectedSeq)
fmt.Printf(" 유효 기간: %s\n", selectedValidity.Format(time.RFC3339))
}
```

IPNS 레코드 캐싱 시스템

```
import (
"sync"
"time"
)
```

```go
// IPNSCache는 IPNS 레코드를 캐시하는 시스템입니다
type IPNSCache struct {
records  map[string]*CachedRecord
mu sync.RWMutex
}

type CachedRecord struct {
Record *ipns.Record
Data []byte
CachedAt time.Time
ExpiresAt time.Time
}

func  NewIPNSCache() *IPNSCache {
return &IPNSCache{
records: make(map[string]*CachedRecord),
}
}

func (c *IPNSCache) Put(name ipns.Name, record *ipns.Record, data []byte) error
c.mu.Lock()
defer  c.mu.Unlock()
// TTL 확인
```

```
ttl, err := record.TTL()
if err != nil {
return err
}
// 유효 기간 확인
validity, err := record.Validity()
if err != nil {
return err
}
now := time.Now()
expiresAt := now.Add(ttl)
// 유효 기간이 TTL보다 짧으면 유효 기간을 사용
if validity.Before(expiresAt) {
expiresAt = validity
}
c.records[name.String()] = &CachedRecord{
Record: record,
Data: data,
CachedAt: now,
ExpiresAt: expiresAt,
}
return nil
}
```

```go
func (c *IPNSCache) Get(name ipns.Name) (*ipns.Record, []byte, bool) {
c.mu.RLock()
defer c.mu.RUnlock()
cached, exists := c.records[name.String()]
if !exists {
return nil, nil, false
}
// 만료 확인
if time.Now().After(cached.ExpiresAt) {
// 만료된 레코드는 백그라운드에서 정리
go c.cleanup(name.String())
return nil, nil, false
}
return cached.Record, cached.Data, true
}

func (c *IPNSCache) cleanup(nameStr string) {
c.mu.Lock()
defer c.mu.Unlock()
delete(c.records, nameStr)
}
```

```go
func (c *IPNSCache) CleanupExpired() int {
c.mu.Lock()
defer c.mu.Unlock()
now := time.Now()
cleaned := 0
for nameStr, cached := range c.records {
if now.After(cached.ExpiresAt) {
delete(c.records, nameStr)
cleaned++
}
}
return cleaned
}

func (c *IPNSCache) Size() int {
c.mu.RLock()
defer c.mu.RUnlock()
return len(c.records)
}

// 캐시 사용 예제
func cacheExample() {
cache := NewIPNSCache()
```

```
// 매니저 생성
sk, _, _ := crypto.GenerateEd25519Key(rand.Reader)
manager, _ := NewIPNSManager(sk)
// 레코드 생성 및 캐시
testPath, _ := path.NewPath("/ipfs/bafkqac3jobxhgidsn5rww4yk")
record, data, _ := manager.UpdateRecord(testPath, 10*time.Second)
err := cache.Put(manager.GetName(), record, data)
if err != nil {
log.Fatal("캐시 저장 실패:", err)
}
fmt.Printf("캐시 크기: %d\n", cache.Size())
// 캐시에서 조회
cachedRecord, cachedData, found := cache.Get(manager.GetName())
if found {
value, _ := cachedRecord.Value()
fmt.Printf("캐시에서 조회: %s\n", value.String())
}
// 만료 대기
fmt.Println("TTL 만료 대기 중...")
time.Sleep(11 * time.Second)
// 만료 후 조회
_, _, found = cache.Get(manager.GetName())
fmt.Printf("만료 후 조회 결과: %v\n", found)
```

```
// 만료된 레코드 정리
cleaned := cache.CleanupExpired()
fmt.Printf("정리된 레코드 수: %d\n", cleaned)
fmt.Printf("정리 후 캐시 크기: %d\n", cache.Size())
}
```

고급 IPNS 활용: 체인 업데이트

```
// IPNSChain은 IPNS 레코드의 체인 업데이트를 관리합니다
type IPNSChain struct {
manager *IPNSManager
history []ChainEntry
mu sync.RWMutex
}

type ChainEntry struct {
Record *ipns.Record
Data []byte
Timestamp time.Time
Value path.Path
Sequence  uint64
}
```

```go
func NewIPNSChain(sk crypto.PrivKey) (*IPNSChain, error) {
manager, err := NewIPNSManager(sk)
if err != nil {
return nil, err
}
return &IPNSChain{
manager: manager,
history: make([]ChainEntry, 0),
}, nil
}


func (c *IPNSChain) Update(newPath path.Path, ttl time.Duration) error {
c.mu.Lock()
defer c.mu.Unlock()
record, data, err := c.manager.UpdateRecord(newPath, ttl)
if err != nil {
return err
}
seq, _ := record.Sequence()
entry := ChainEntry{
Record: record,
Data: data,
Timestamp: time.Now(),
```

```
Value: newPath,
Sequence: seq,
}
c.history = append(c.history, entry)
return nil
}

func (c *IPNSChain) GetLatest() (*ChainEntry, bool) {
c.mu.RLock()
defer c.mu.RUnlock()
if len(c.history) == 0 {
return nil, false
}
return &c.history[len(c.history)-1], true
}

func (c *IPNSChain) GetHistory() []ChainEntry {
c.mu.RLock()
defer c.mu.RUnlock()
// 복사본 반환
history := make([]ChainEntry, len(c.history))
copy(history, c.history)
return history
```

```
}

func (c *IPNSChain) GetName() ipns.Name {
return  c.manager.GetName()
}

// 체인 업데이트 예제
func  chainUpdateExample() {
sk, _, _ := crypto.GenerateEd25519Key(rand.Reader)
chain, _ := NewIPNSChain(sk)
fmt.Printf("IPNS 체인 이름: %s\n", chain.GetName().String())
// 여러 번 업데이트
paths := []string{
"/ipfs/bafkqac3jobxhgidsn5rww4yk",
"/ipfs/bafkreifjjcie6lypi6ny7amxnfftagclbuxndqonfipmb64f2km2devei4",
"/ipfs/bafkreihdwdcefgh4dqkjv67uzcmw7ojee6xedzdetojuzjevtenxquvyku",
}
for  i, pathStr := range  paths {
p, _ := path.NewPath(pathStr)
err := chain.Update(p, 30*time.Second)
if  err != nil {
log.Fatal(err)
}
```

```
fmt.Printf("업데이트 %d: %s\n", i+1, pathStr)
time.Sleep(1 * time.Second) // 시간 차이를 위해
}
// 히스토리 출력
fmt.Println("\n업데이트 히스토리:")
history := chain.GetHistory()
for i, entry := range history {
fmt.Printf("%d. 시퀀스: %d, 값: %s, 시간: %s\n",
i+1, entry.Sequence, entry.Value.String(),
entry.Timestamp.Format("15:04:05"))
}
// 최신 레코드 확인
if latest, ok := chain.GetLatest(); ok {
fmt.Printf("\n최신 레코드: %s (시퀀스: %d)\n",
latest.Value.String(), latest.Sequence)
}
}


func main() {
fmt.Println("=== IPNS 이름 생성 ===")
createIPNSName()
fmt.Println("\n=== IPNS 레코드 생성 ===")
createIPNSRecord()
```

```
fmt.Println("\n=== 다양한 키 타입 ===")
createIPNSNamesWithDifferentKeys()
fmt.Println("\n=== 이름 파싱 ===")
parseIPNSNames()
fmt.Println("\n=== CID에서 이름 생성 ===")
createIPNSNameFromCID()
fmt.Println("\n=== IPNS 매니저 예제 ===")
ipnsManagerExample()
fmt.Println("\n=== 레코드 비교 예제 ===")
recordComparisonExample()
fmt.Println("\n=== 캐시 예제 ===")
cacheExample()
fmt.Println("\n=== 체인 업데이트 예제 ===")
chainUpdateExample()
}
```

모범 사례

1. 키 관리

- Ed25519 키 사용 권장 (성능과 보안 우수)
- 개인키 안전한 저장 및 백업
- 키 순환 정책 수립

2. 레코드 설계

- 적절한 TTL 설정 (너무 짧으면 네트워크 부하, 너무 길면 업데이트 지연)
- 유효 기간을 TTL의 10배 이상으로 설정
- 시퀀스 번호 단조 증가 보장

3. 성능 최적화

- 레코드 캐싱 시스템 구축
- 배치 업데이트로 네트워크 부하 감소
- 적절한 만료 시간 설정

4. 보안 고려사항

- 레코드 검증 필수
- 개인키 보안 관리
- 타임스탬프 검증

문제 해결

일반적인 오류와 해결책

1. **레코드 검증 실패**

- 서명 확인: 올바른 개인키 사용 여부
- 시간 동기화: 시스템 시간 정확성 확인
- 레코드 형식: 직렬화/역직렬화 오류 확인

2. **만료된 레코드**

- 유효 기간 연장

- 자동 갱신 시스템 구축
- 적절한 TTL 설정

3. **시퀀스 번호 문제**

- 단조 증가 보장
- 동시성 제어
- 상태 동기화

4. **네트워크 전파 지연**

- DHT 라우팅 최적화
- 여러 노드에 동시 발행
- 적절한 재시도 로직

5. **키 복구 문제**

- 안전한 백업 시스템
- 키 분할 저장
- 복구 절차 문서화